MINI PROJECT REPORT

AI23521 - BUILD AND DEPLOYMENT OF MACHINE LEARNING APPLICATIONS

STUDENT PERFORMANCE ANALYSIS, PREDICTION, AND DEPLOYMENT SYSTEM



Submitted by

SAJIN REYANS J

241501176

Department of Artificial Intelligence and Machine Learning

RAJALAKSHMI ENGINEERING COLLEGE

Academic Year: 2024-2028

CONTENTS

S. No.	Section
1	Abstract
2	Introduction
3	Problem Statement and Objectives
4	Scope
5	Tools and Technologies
6	Dataset Description
7	System Architecture
8	Methodology and Algorithms
9	Model Evaluation
10	Flask API, MLOps Artifacts and Docker
11	Results and Discussion
12	Advantages and Limitations
13	Future Enhancements
14	Conclusion
15	References
Appendix A	Source Code
Appendix B	Execution Instructions

1. ABSTRACT

This project presents an end-to-end machine learning and MLOps workflow for analysing, predicting and deploying workforce-attrition outcomes. Using a dataset of employee records with demographic, job, compensation, work-condition and engagement attributes, the system classifies whether an employee is likely to leave the organisation (Attrition = Yes) or stay (Attrition = No). The workflow covers exploratory data analysis and statistical profiling, preprocessing and feature engineering, class-imbalance handling with SMOTE, benchmarking of five classical classifiers, a feedforward neural network and a one-dimensional convolutional neural network, K-Means clustering with PCA and t-SNE, a Generative Adversarial Network (GAN) for data augmentation, hyperparameter tuning with GridSearchCV and RandomizedSearchCV, and model serialisation with joblib.

The best model is exposed through a Flask REST API with health-check, prediction and metrics endpoints, tested under simulated concurrent load, and packaged with generated deployment files (app.py, requirements.txt, a multi-stage Dockerfile, .dockerignore and deploy.sh). The dataset is synthetic and intended for educational demonstration; results must not be interpreted as validated predictions about real employees.

2. INTRODUCTION

Employee attrition is costly: it removes accumulated knowledge, raises recruitment and training expenses, and disrupts teams. Organisations therefore try to understand which workplace conditions are associated with resignations. Structured HR data such as overtime, time since last promotion, satisfaction ratings and commuting distance can be analysed with machine learning to find such patterns.

This project brings the topics of the Build and Deployment of Machine Learning Applications course into one workflow, from data profiling through model evaluation to API deployment. It compares classical algorithms with neural-network approaches, adds an unsupervised view of the workforce, demonstrates synthetic-data generation, and shows how a trained model can be served and containerised. The purpose is academic learning and algorithm practice, not automated HR decision-making.

3. PROBLEM STATEMENT AND OBJECTIVES

Problem statement: Develop a demonstration system that uses employee-related features to predict attrition, compares multiple algorithms, explores workforce patterns, tunes model parameters, and provides real-time predictions through a REST API.

- Load and profile the employee attrition dataset (shape, data types, memory usage, unique values and missing-value percentages).
- Produce at least six exploratory visualisations using Matplotlib and Seaborn.

- Impute, encode and scale the data, address class imbalance with SMOTE, and create stratified train and test splits.
- Train and compare SVM, Random Forest, Gradient Boosting, Logistic Regression and Decision Tree models using Accuracy, Precision, Recall, F1-score and ROC-AUC.
- Build and train a deep feedforward network and a 1D CNN with dropout and early stopping.
- Apply K-Means (Elbow and Silhouette analysis), PCA and t-SNE to explore workforce segments.
- Build a GAN from scratch to generate synthetic employee records.
- Tune models with GridSearchCV and RandomizedSearchCV and save the best model as employee_model_v1.pkl.
- Serve the model through a Flask API, test it with concurrent requests, and generate Docker and deployment files.

4. SCOPE OF THE PROJECT

The project is designed for coursework and runs in Google Colab on a CSV dataset located at /content/sample_data/employee_attrition_dataset.csv. It demonstrates exploratory analysis, supervised learning, unsupervised learning, deep learning, generative modelling, model tuning, serialisation and deployment. The provided data is simulated. A real deployment would require representative data, employee consent and privacy safeguards, careful validation, fairness testing across protected groups, and human oversight, because attrition predictions can influence people's careers.

5. TOOLS AND TECHNOLOGIES

Tool / Library	Purpose
Python	Main programming language
Google Colab	Cloud notebook environment
Pandas and NumPy	Data manipulation and numerical processing
Matplotlib and Seaborn	Statistical charts and visualisations
Scikit-learn	Preprocessing, classifiers, clustering, PCA, t-SNE, evaluation and tuning
imbalanced-learn	SMOTE oversampling for class imbalance
TensorFlow / Keras	FNN, 1D CNN and GAN implementations; TensorBoard-style logging
Joblib	Saving and loading the trained model
Flask	REST API for inference and metrics
Requests and threading	Concurrent API test simulation
Docker	Containerisation using a multi-stage Dockerfile

6. DATASET DESCRIPTION

The dataset contains 500 synthetic employee records and 30 columns. EmployeeID is an identifier and is excluded from modelling. Attrition is the target. The dataset has no missing values and is class-imbalanced: 117 employees are labelled Yes (23.4%) and 383 are labelled No (76.6%).

The records were generated programmatically. Attrition was assigned using a probabilistic rule in which the likelihood of leaving rises with overtime, low work-life balance or job satisfaction, long gaps since promotion, short tenure, frequent job changes, low salary hikes, long commutes and frequent travel, and falls with higher age, higher salary and more training. The model scores therefore reflect this simulated process and not real-world employee behaviour.

Column	Description	Role
EmployeeID	Unique employee identifier	Identifier (dropped)
Age	Age in years	Input
Gender	Male, Female or Other	Input
MaritalStatus	Single, Married or Divorced	Input
Education	High School, Bachelor's, Master's or PhD	Input
City	Work location city	Input
Department	Sales, Engineering, HR, Finance, Marketing, Operations, Customer Support or IT	Input
JobRole	Job title within the department	Input
JobLevel	Seniority level (1 to 3)	Input
TotalWorkingYears	Total professional experience in years	Input
YearsAtCompany	Tenure at the current company	Input
YearsSinceLastPromotion	Years since last promotion	Input
YearsWithCurrManager	Years under the current manager	Input
NumCompaniesWorked	Number of previous employers	Input
AnnualSalaryINR	Annual salary in Indian rupees	Input
LastHikePercent	Most recent salary hike (%)	Input
OverTime	Whether the employee works overtime (Yes/No)	Input
WorkMode	Onsite, Hybrid or Remote	Input
BusinessTravel	None, Rarely or Frequently	Input
DistanceFromHomeKm	Commute distance in kilometres	Input
TrainingsLastYear	Number of trainings attended last year	Input
PerformanceRating	Performance rating (1 to 5)	Input
JobSatisfaction	Satisfaction rating (1 to 4)	Input

Column	Description	Role
EnvironmentSatisfaction	Work-environment rating (1 to 4)	Input
RelationshipSatisfaction	Workplace-relationship rating (1 to 4)	Input
WorkLifeBalance	Work-life balance rating (1 to 4)	Input
AbsentDaysLastYear	Days absent in the last year	Input
LeavesTaken	Leaves taken in the last year	Input
TeamSize	Number of people in the team	Input
Attrition	Yes if the employee left, otherwise No	Target

7. SYSTEM ARCHITECTURE

- Data source: load employee_attrition_dataset.csv from Colab storage.
- Profiling and EDA: summarise the data and save exploratory plots.
- Preprocessing: impute, encode, split into stratified train and test sets, scale features and apply SMOTE to the training set only.
- Supervised learning: benchmark five classical models and two neural networks.
- Unsupervised learning: K-Means on scaled features with PCA and t-SNE projections.
- Generative modelling: train a GAN on scaled features and compare synthetic and original distributions.
- Tuning and serialisation: run GridSearchCV and RandomizedSearchCV, pick the best pipeline and save it with joblib.
- Deployment: serve the model through Flask, test concurrency, and generate Docker and deployment files.

8. METHODOLOGY AND ALGORITHMS

8.1 Exploratory data analysis and statistical profiling

The dataset is loaded with Pandas and profiled using shape, data types, memory usage, unique-value counts per column, descriptive statistics and the percentage of missing values per column. At least six visualisations are generated and saved as image files.

Visualisation	Purpose
Correlation heatmap	Shows linear relationships among numerical features and with the encoded target
Target distribution bar plot	Reveals the class imbalance between Yes and No
Histograms with KDE curves	Show the distribution and skew of numerical features
Boxplots	Detect outliers in salary, tenure and absence
Pairwise scatter plots	Show relationships among selected key features coloured by attrition

Visualisation	Purpose
Violin plots by attrition	Compare feature distributions between employees who left and stayed

8.2 Data preprocessing and feature engineering

Missing values are handled with a median imputer for numerical columns and a most-frequent imputer for categorical columns. The supplied dataset has no missing values, but the imputation step is retained so that the pipeline is robust to incomplete data. Binary categories such as OverTime and the target Attrition are label-encoded, ordered categories such as Education are mapped to ordinal codes, and nominal categories such as Department, JobRole, City, WorkMode, BusinessTravel, Gender and MaritalStatus are one-hot encoded.

An 80:20 stratified split with a fixed random seed preserves the class ratio in both subsets. StandardScaler is applied for distance-based and neural models and MinMaxScaler for the GAN. Both scalers are fitted on the training data only. SMOTE is then applied to the training set only, so that synthetic minority samples never leak into the test set and evaluation reflects the original class distribution.

8.3 Classical machine learning models

Model	Role in the benchmark
Support Vector Machine	Margin-based classifier; requires scaled features
Random Forest	Ensemble of decision trees; provides feature-importance scores
Gradient Boosting	Sequential ensemble that corrects earlier errors (XGBoost, LightGBM or scikit-learn)
Logistic Regression	Interpretable linear baseline with probability outputs
Decision Tree	Rule-based model whose splits can be inspected

Each model is evaluated on the held-out test set using Accuracy, Precision, Recall, F1-score and ROC-AUC. Comparative bar charts, ROC curves and Precision-Recall curves are saved for all models. Because attrition is the minority class, Recall and F1-score for the Yes class are more informative than accuracy alone. Random Forest feature-importance scores describe the fitted model and do not establish that a feature causes attrition.

8.4 Deep learning models

The feedforward neural network uses several dense hidden layers with ReLU activation, dropout regularisation and a sigmoid output, trained with the Adam optimiser and binary cross-entropy. Early stopping monitors validation loss and restores the best weights, and training metrics are logged per epoch in a TensorBoard-compatible format.

The one-dimensional CNN treats the ordered feature vector as a sequence, applies Conv1D and pooling layers followed by dense layers with dropout, and is trained under the same settings.

The CNN is included for syllabus coverage and is not assumed to be better than tree-based models on tabular data. Training and validation loss and accuracy curves are plotted for both networks.

8.5 K-Means, PCA and t-SNE

K-Means is applied to standardised features. The Elbow method (inertia against the number of clusters) and the Silhouette score are used to select the number of clusters. PCA projects the data to two and three components, and the explained-variance ratios are plotted. t-SNE provides a non-linear two-dimensional embedding. Clusters are discovered without using the target labels, so they do not necessarily correspond to Attrition = Yes or No, although the attrition rate within each cluster can be inspected.

8.6 Generative Adversarial Network

The GAN contains a generator that maps random latent vectors to scaled employee feature records and a discriminator that learns to distinguish original from generated records. The two networks are trained in alternating steps on MinMax-scaled features. Generated records are inverse-transformed and compared with original data through scatter plots. With a small dataset and limited training, the synthetic data may not reproduce the original distribution reliably, so it is used for demonstration rather than as a replacement for real data.

8.7 Hyperparameter tuning and serialisation

GridSearchCV and RandomizedSearchCV are applied across several model pipelines using stratified cross-validation on the training set, with the search scored on a metric suited to imbalanced data such as F1-score or ROC-AUC. Cross-validation scores, detailed classification reports and annotated confusion-matrix heatmaps are produced for the tuned models. The best-performing pipeline is evaluated once on the test set and serialised with joblib as employee_model_v1.pkl. Keeping tuning on the training data keeps the final test evaluation separate from parameter selection.

9. MODEL EVALUATION

Accuracy measures the proportion of correct predictions. Precision measures how many predicted leavers actually left; Recall measures how many actual leavers were identified; F1-score combines the two; and ROC-AUC measures how well the model ranks leavers above stayers across all thresholds. A confusion matrix shows the distribution of correct and incorrect predictions. The notebook prints the actual scores when executed.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
SVM	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output
Random Forest	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Gradient Boosting	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output
Logistic Regression	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output
Decision Tree	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output
Feedforward NN	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output
1D CNN	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output
Best tuned model	Fill from output	Fill from output	Fill from output	Fill from output	Fill from output

Actual metric values should be copied from the executed notebook; they should not be estimated or invented. Because the labels follow a generated rule, high scores on this dataset do not establish real-world predictive validity.

10. FLASK API, MLOPS ARTIFACTS AND DOCKER

10.1 REST API endpoints

Method	Endpoint	Purpose
GET	/	Health check; confirms the API is running
POST	/predict	Validates the JSON input and returns the predicted attrition class with a probability
GET	/metrics	Returns a summary of the saved model's performance metrics

10.2 Example request

The /predict endpoint accepts a JSON object containing the employee's feature values. The server validates that required fields are present and have valid types and ranges, and returns an error with HTTP status 400 otherwise.

```
POST /predict
Content-Type: application/json
{
  "Age": 36, "Department": "Sales", "JobLevel": 2,
  "YearsAtCompany": 3.5, "YearsSinceLastPromotion": 3.0,
  "AnnualSalaryINR": 900000, "OverTime": "Yes",
  "WorkLifeBalance": 2, "JobSatisfaction": 2
  ... (remaining features)
}
```

Example response structure (the values depend on the fitted model):

```
{
  "attrition_prediction": "Yes",
```

```
  "attrition_probability": 0.00
}
```

10.3 Concurrent request testing

A test script uses the requests library and Python threads to send many simultaneous prediction requests to the running API. It records the HTTP status codes and response times so that the stability of the service under concurrent load can be checked.

10.4 Generated deployment files

File	Purpose
app.py	Flask application that loads employee_model_v1.pkl and serves the endpoints
requirements.txt	Python dependencies for the container
Dockerfile	Multi-stage build: dependencies are installed in a builder stage and only the runtime files are copied into a slim final image; exposes port 5000
.dockerignore	Excludes notebooks, plots, caches and data from the build context
deploy.sh	Shell script that builds the image, runs the container and checks the health endpoint

```
docker build -t employee-attrition .
docker run -p 5000:5000 employee-attrition
```

Colab can generate the files and test the API locally, but a Docker daemon is generally not available in a standard Colab runtime, so the Docker commands should be run on a machine with Docker installed.

11. RESULTS AND DISCUSSION

The notebook is designed to display the dataset summary, the EDA visualisations, the model comparison charts, ROC and Precision-Recall curves, confusion-matrix heatmaps, neural-network training curves, the K-Means Elbow and Silhouette plots, PCA and t-SNE projections, GAN comparison plots, the tuned-model results and the API test output. Insert screenshots from the actual Colab output into this section when preparing the final submitted copy.

The results illustrate the different roles of the algorithms: Logistic Regression gives an interpretable baseline, Decision Trees provide visible rules, Random Forest and Gradient Boosting combine many trees, SVM uses a margin-based boundary, neural networks learn non-linear combinations, K-Means and PCA reveal unlabeled structure, and the GAN shows how synthetic records can be generated. SMOTE is expected to improve the recall of the minority class, which should be confirmed from the printed metrics rather than assumed.

12. ADVANTAGES AND LIMITATIONS

12.1 Advantages

- Combines data profiling, supervised, unsupervised, deep and generative learning in one workflow.
- Handles class imbalance correctly by applying SMOTE only to the training data.
- Compares five classical models and two neural networks with several metrics.
- Includes tuning, serialisation, an API with a metrics endpoint, concurrency testing and container files.

12.2 Limitations

- The dataset is synthetic with 500 records, not collected from real employees.
- High scores may reflect the rule used to generate the labels.
- CNN and GAN are included for learning objectives and may not suit small tabular data.
- GAN-generated records may be unrealistic.
- The API is tested locally and is not publicly hosted by default.
- Docker execution requires a Docker-enabled environment.

13. FUTURE ENHANCEMENTS

- Use a representative, ethically collected and anonymised employee dataset.
- Test the model on data from a different period or organisation.
- Add model explainability with SHAP or LIME.
- Evaluate calibration, fairness and performance across relevant groups.
- Add experiment tracking, model versioning, automated retraining and monitoring for data drift.
- Build a dashboard of attrition risk and workforce analytics.
- Keep predictions advisory and avoid using them as the sole basis for consequential employment decisions.

14. CONCLUSION

The project demonstrates a complete machine learning lifecycle on an employee attrition dataset: profiling, preprocessing, imbalance handling, benchmarking of classical and deep models, clustering and dimensionality reduction, GAN-based augmentation, hyperparameter tuning, serialisation and deployment through a Flask API with container files. It gives practical experience with the course syllabus. Since the dataset is synthetic, the methods should be validated on appropriate real-world data before any conclusions are drawn about actual employee attrition.

15. REFERENCES

- Scikit-learn documentation: https://scikit-learn.org/stable/
- imbalanced-learn documentation: https://imbalanced-learn.org/stable/
- Pandas documentation: https://pandas.pydata.org/docs/
- Matplotlib documentation: https://matplotlib.org/stable/
- Seaborn documentation: https://seaborn.pydata.org/
- TensorFlow documentation: https://www.tensorflow.org/guide
- Flask documentation: https://flask.palletsprojects.com/
- Docker documentation: https://docs.docker.com/

APPENDIX A: SOURCE CODE

Paste the final combined project script (employee_attrition_pipeline.py) here before submission.

APPENDIX B: EXECUTION INSTRUCTIONS

1. Download employee_attrition_dataset.csv.
2. Open Google Colab and create a new Python notebook.
3. Upload the dataset to /content/sample_data/.
4. Paste the project script into a cell and run it.
5. Wait for training, visualisations, API testing and file creation to finish.
6. Record the actual scores and take screenshots of the generated outputs.
7. Download employee_model_v1.pkl, app.py, requirements.txt, Dockerfile, .dockerignore and deploy.sh from the Colab Files panel.
8. Run the Docker commands only in an environment where Docker is installed.